AI INFRA ・PAPER STUDY / 03

Megatron Core MoE 训练系统

论文深度解析

Scalable Training of Mixture-of-Experts Models with Megatron Core

从并行映射、激活生命周期到动态 CUDA Graph，理解稀疏模型如何真正转化为训练吞吐。

核心判断

这不是一篇单一算子或新路由算法论文，而是一份 MoE 训练系统的协同设计报告。最值得精读的是 Parallel Folding、通信关键路径重排，以及为动态专家负载设计的内存与执行机制。

原论文：NVIDIA 团队，arXiv:2603.07685v2；88 页，42 幅图。
本解析基于 arXiv HTML 正文与附录、官方仓库说明及相关 PR。
核对日期：2026-10-19。性能数字为作者报告，未独立复跑。

目录

一、先给结论：它真正解决的不是“多加几张卡” 2

二、先跟着一个 token 走完 MoE 层 2

三、Parallel Folding：同一批 GPU，可以有两套布局 4

四、显存优化：不只是把数据类型变小 5

五、通信优化：真正重要的是暴露在关键路径上的时间 7

六、计算效率：Grouped GEMM 和 CUDA Graph 各解决一层问题 8

七、动态 MoE 怎样进入 CUDA Graph：三个问题，三层答案 9

八、FP8 与 NVFP4：精度、布局、通信必须一起看 10

九、长上下文与 RL：瓶颈会换位置 12

十、主实验：数字很强，但有三个不可省略的条件 13

十一、严谨性与可复现性：我会追问什么? 14

十二、怎样把论文变成一个实际的优化项目? 15

十三、最后把整篇论文收束成一个系统观点 17

附录 A：术语速查 18

附录 B：来源与核对范围 19

一、先给结论：它真正解决的不是“多加几张卡”

这篇技术报告的中心问题是：**MoE 降低了每个 token 激活的参数量，为什么训练系统仍然可能很慢，甚至比同等计算量的稠密模型更难优化？**

答案不是某个 GPU 指令不够快，而是三种资源需求不再同步增长：参数和优化器状态跟随全部专家增长；有效计算主要跟随被选中的专家增长；通信、调度和不规则张量处理却会随着专家划分、路由分布和拓扑变化而恶化。报告将它们组织为显存、通信、计算效率三类瓶颈。[S1，§1–4]

我的整体评价是：**它的工程解释价值高于“单个新算法”的价值**。不应把 Grouped GEMM、重计算、FP8 等所有组成部分都当成这篇报告首次发明的技术；Parallel Folding 已有 2025 年的独立论文。这里更重要的是展示这些方法怎样组合、互相限制，为什么同一模型在不同机器上要采用不同方案。[S2]

全文可以围绕下面这条因果链阅读：

> 先消除不必要的保存，获得显存余量；再选择合理的并行布局；随后用额外缓冲、跨 microbatch 调度和图执行降低暴露开销；最后重新检查瓶颈是否转移。

本解析区分三种内容：**原文结果**用来源编号与原章节定位；**解释性推导**说明假设，不当作作者实验；**代码状态核对**只描述实际查看的文档或 PR，不等同于完整源码审计。原文主表使用强制均衡路由，这是解读所有 headline 数字的前提。[S1，§8.2]

1.1 阅读导航

阅读目标	最值得看的部分	要回答的问题
理解大规模训练布局	本解析三；原文 §3	Attention 与专家为什么不该共用一种切分方式?
做算子和显存优化	本解析四、六、八	哪些张量不必保存，哪些低精度开销会抵消 GEMM 收益?
做 MoE 通信调度	本解析五	如何重排依赖，让通信不再处于关键路径?
研究动态训练运行时	本解析七	动态 token 数怎样与 CUDA Graph、静态地址共存?
复现和评估结果	本解析十至十二	强制均衡、并行配置变化、原型状态会怎样影响结论?

二、先跟着一个 token 走完 MoE 层

2.1 参数多，不等于每个 token 都计算全部参数

忽略 shared expert 与 bias，用两层 MLP 表示一个专家：

$$y(x) = \sum_{e \in \mathcal{T}_k(x)} p_e(x)\, W_{2,e}\, \phi(W_{1,e} x).$$

这里 E 是专家总数，k 是每个 token 选中的专家数，$\mathcal{T}_k(x)$ 是选择集合。存储需要容纳全部 E 个专家，但单个 token 的专家计算只经过 k 个。[S1，§2]

报告使用 DeepSeek-V3-685B 作为代表，其中 685B 包含主模型和 MTP 部分；不能不加区分地把它与常见的 671B 主模型口径混为一谈。报告列出的每 token 激活参数约为 37B，用于说明总参数与实际计算的差距。[S1, §1、§9.2]

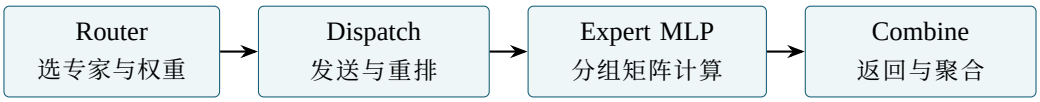
但 “仅激活约 37B” 不表示训练时只需要保存这 37B 的权重。其他 token、其他 batch 会选择不同专家，优化器还需要跟踪这些参数。因此，MoE 的核心困难是：**显存更接近大模型，单次计算却更接近小模型，通信和管理开销相对更突出**。

2.2 一次前向包含 dispatch，也包含 combine

正确的逻辑顺序是：

$$\text{Router} \rightarrow \text{Dispatch} \rightarrow \text{Expert MLP} \rightarrow \text{Combine}.$$

Router 得到专家索引和权重；dispatch 将 token 送到专家所在 rank，并组织为适合计算的布局；专家执行各自的 FC1、激活和 FC2；combine 将结果送回原 token 所在位置并聚合。反向传播还要执行对应的梯度通信。**不能把 dispatch 说成前向、combine 说成反向**。[S2，§3.3；S3]



前向内部已包含两次 token 交换；反向还要传播梯度。
图表示逻辑依赖，不表示每段真实耗时。

2.3 为什么增大 EP 并不自动让每张卡的 token 数减少？

做一个简化推导。假设一个 EP 组有 P 个 rank，每个 rank 输入 T 个互不重复的 token，均匀 top-k 路由到 E 个专家，且不考虑 ETP 复制。那么每个专家平均处理：

$$\bar{M}_e = \frac{PTk}{E}.$$

每张卡有 E/P 个专家，所以平均接收的 token–expert 分配总数是：

$$\frac{E}{P} \cdot \bar{M}_e = Tk.$$

在固定每 rank 输入 T 的假设下，EP 增大减少了本地专家权重数量，却不一定减少总激活工作量。每个本地专家反而可能聚合更多 token。这也解释了为何 EP 常能改善单个专家 GEMM 的形状，但不能被当成 “所有内存都除以 EP” 的开关。

同样，先忽略目标 rank 去重、元数据和不同传输精度，单 rank 前向 dispatch 加 combine 的有效载荷可粗略写为：

$$V_{\text{fwd}} \approx 2TkHb\frac{P-1}{P},$$

H 为 hidden size，b 为每元素字节数。这个式子用于解释 “远端比例”，不是 DeepEP/HybridEP 的精确流量公式：同一个 token 的多个目标专家若位于同一目的地，专用 dispatcher 可以去重；不同阶段还可能使用不同 dtype。[S1，§4.2；S3]

由式子可见，在固定本地输入下，远端比例趋于 1，不会无限线性增长。**真正剧烈的变化往往发生在 EP 跨出高速互联域时，而不是 EP 数字本身翻倍时**。

三、Parallel Folding：同一批 GPU，可以有两套布局

3.1 为什么 Attention 和 Expert 要区别对待?

Attention 需要跨 token 交换信息，长序列尤其需要 CP；稠密投影也可能因为参数或激活容量需要 TP。Expert MLP 则主要独立处理路由后的 token，继续在它内部使用较大的 TP，可能把已经不大的矩阵进一步切碎，并增加额外 collectives。[S2，§3.2]

传统布局把 EP 放在 DP 的子组内，这会让专家并行度受 DP 约束。Parallel Folding 的做法是：**Attention 和 MoE 分别解释同一批 rank 的并行坐标，而不是把所有并行维度机械相乘**。[S2；S3]

设总 GPU 数为 W：

$$W = TP \times CP \times DP \times PP = ETP \times EP \times EDP \times PP.$$

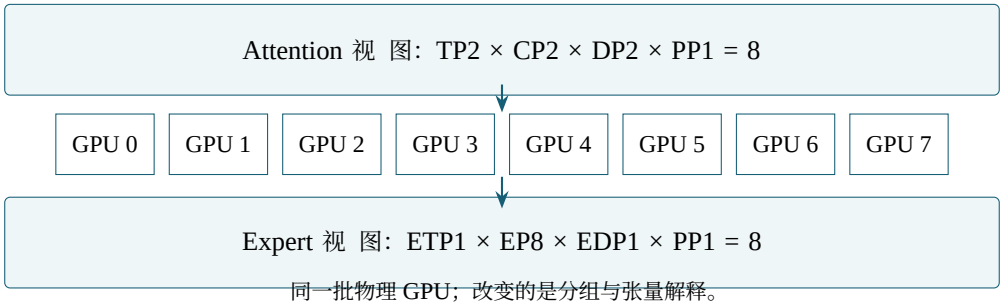
ETP 是专家内部张量并行，EDP 是同一专家的复制/数据并行维度。两套布局仍要保持兼容的 pipeline 划分、合法的整除关系和正确的数据重分布，绝不是任意填写几个乘积相等的数字就必然能运行。

3.2 用 8 张卡理解，不必先看 256 卡

考虑以下**解释性示例**，不代表 685B 模型能装入这 8 张卡：

模块	TP / ETP	CP / EP	DP / EDP	PP	总卡数
Attention	TP=2	CP=2	DP=2	1	8
Expert MLP	ETP=1	EP=8	EDP=1	1	8

Attention 阶段可以用 TP2、CP2 处理两份数据；到 MoE 阶段，这些 rank 上的 token 进入一个 EP8 域，专家内部不再做 TP。**没有增加第 9 张卡，也没有把 CP2 再乘到 EP8 上**。



同一批物理 GPU；改变的是分组与张量解释。
Dispatcher 负责两种布局之间的衔接。

官方文档还给出一个 256 卡例子：Attention 使用 TP4、CP2、DP8、PP4；Expert 使用 ETP1、EP64、EDP1、PP4。原来局限于 DP8 的 EP，可以扩展到 64。[S3]

3.3 EP 与 EDP 混淆，会直接导致梯度同步错误

EP 里的不同 rank 可能拥有**不同专家**；EDP 里的 rank 则拥有**同一专家的不同副本**。同一个专家的副本需要聚合来自不同数据的梯度，不同专家的梯度不能简单互相求和。

因此，看到某个专家参数不走普通 dense DP 的 all-reduce，不能推出 “它不需要同步”。可能只是同步发生在专门的 EDP 组。FSDP 对专家参数的分片范围也必须遵守这个区别。[S1，§4.1.7]

最值得带走的思想：并行布局应跟随子模块的计算结构，而不是强迫整个 Transformer 使用同一种拓扑。Folding 解除约束，但并不消除通信、分片边界或数值一致性问题。

四、显存优化：不只是把数据类型变小

4.1 参数不是唯一大头，激活生命周期更关键

报告对一个特定 DeepSeek-V3 BF16 配置给出以下分解：256 GPU、PP4、VPP4、EP64。请把这些数字当作该配置的样本，而不是模型在任意布局下的固有占用。[S1，表 3]

显存组成	每 GPU 占用
模型权重与梯度	36.4 GB
主权重与优化器状态	32.1 GB
激活	131.0 GB
合计	199.5 GB

这里激活占比约为 $131/199.5 \approx 65.7\%$。这个例子说明：继续只盯权重分片，可能根本没有抓住最大问题。激活还受多路专家分发、padding、pipeline 中未完成的 microbatch 数量和实际路由偏斜影响。

更重要的是，省出的显存不是只能留空。它能换成更大的 microbatch、更低的模型并行度、通信重叠缓冲或 CUDA Graph 的静态存储。**显存优化因此可能间接提高吞吐，而不只是防止 OOM**。[S1，§4.1.8]

4.2 一个很漂亮的小改动：把路由概率移到 FC2 前

令 $h_e = \phi(W_{1,e}x)$。标准实现通常先计算专家输出，再乘概率：

$$y_e = p_e(W_{2,e}h_e).$$

如果 FC2 没有 bias，它等价于：

$$y_e = W_{2,e}(p_e h_e).$$

关键不是少做一次乘法，而是改变自动求导需要跨前后向保存什么。旧路径要为概率梯度保留专家的完整输出；新路径可以利用已经为激活反向保存的 pre-activation，在 fused backward 中恢复所需值，避免额外长期保存完整专家输出。[S1，§4.1.2]

设 $g = \partial \mathcal{L}/\partial y_e$，则有：

$$\frac{\partial \mathcal{L}}{\partial p_e} = g^\top W_{2,e} h_e = (W_{2,e}^\top g)^\top h_e.$$

这个梯度恒等式说明，概率梯度不必依赖 “完整专家输出一直活着” 这一具体实现方式。需要什么值、这些值是否已为其他反向操作保存，才决定真实的内存收益。报告在上述配置下给出的节省约为 **26.3 GB/GPU**。[S1，表 3、§4.1.2]

这是一类特别适合算子优化人员学习的方法：**先改中间结果的生命周期，再用融合把额外的数据访问和轻量重算消掉**。

这里有两个边界。第一，有 bias 时要保持 $p_e b_e$ 的正确处理，不能直接套用纯线性恒等式。第二，不能把 p_e 随意移过非线性函数：一般 $p\phi(z) \neq \phi(pz)$。

还有一个与你关心的确定性特别相关的区别：**实数数学等价不代表浮点逐位等价**。原来先做 GEMM 再乘标量，现在先缩放输入再做 GEMM，舍入与量化边界都可能变化；涉及 FP8 时，动态范围和 scale 也可能改变。需要分别验证数学语义、数值容差和 bitwise 目标，不能只凭代数式判定 “确定性完全保持”。

4.3 重计算要按 “释放多少、关键路径多花多少” 选择

整层 checkpointing 会重新执行大量计算。MoE 的额外危险是：若重算范围跨过专家通信，还可能把 dispatch/combine 重新跑一遍。因此，它增加的不只是 GEMM FLOPs。[S1，§4.1.4]

报告采用模块级选择性重计算与输出丢弃，在上述 DeepSeek 配置下给出：[S1，表 4]

重计算对象	每 GPU 节省
MLA up-projection	30.4 GB
SwiGLU	3.8 GB
LayerNorm	8.2 GB
该表合计	42.4 GB

理解这个策略，可以用一个**分析性指标**：

$$\rho_i = \frac{\Delta M_i}{\Delta t_{i,\text{exposed}}}.$$

分子是降低的峰值显存, 分母是实际暴露在训练关键路径上的额外时间。注意这不是单独算子的 “字节/FLOPs”，因为通信、并发窗口和保存时机都会影响分母。

表格中的内存收益也不能与其他技术无条件相加。两种方法可能释放同一个张量，或改变峰值出现的位置；必须在组合配置上重新测 peak memory。

4.4 Offload：把昂贵重算换成可重叠的数据搬运

细粒度 offload 可以只迁移适合的模块输入：前向在后续计算期间做 D2H，反向提前 H2D reload。层间错开预取，还需要正确匹配 PP/VPP 的 microbatch 与虚拟阶段。[S1，§4.1.5；S3]

可用一个理想化判断理解它：当可并发的独立计算窗口长于搬运时间，迁移可以少暴露延迟；反之，就会在等待 activation 时停住。但 Copy Engine 与 Compute Engine 分开，并不表示 HBM、链路和缓存争用消失。

所以，“offload 没有代价” 不成立。更准确的说法是：**有时迁移代价能被隐藏；有时它带来的显存余量允许更好的并行配置，抵消甚至超过搬运开销**。报告表 5 的 Qwen 案例改变了 TP 和 EP，因此其增益不是 offload 单开关的纯收益。

FSDP、低精度优化器状态与 activation offload 解决的是不同内存对象。尤其 FSDP 对专家参数应在 EDP 维度分片，而不是把不同专家当成同一份参数做全局分片。[S1，§4.1.6–7]

五、通信优化：真正重要的是暴露在关键路径上的时间

5.1 专用 dispatcher 不等于 “换一个 All-to-All API”

普通路径往往把 token 复制、排列、通信、还原、加权归约分成多段。DeepEP/HybridEP 的价值包括按 token 组织传输、减少同一目的地上的重复发送、分层跨节点/节点内搬运，并将 combine 的部分归约与通信融合。[S1，§4.2.2；S3]

HybridEP 还面向 TMA、IBGDA、MNNVL 等硬件机制优化。论文在 GB200/GB300 NVL 系统采用 HybridEP，在 H100 跨节点系统采用 DeepEP。这个选择不能直接推广成某个后端在所有 GPU 和网络上更好。[S1，附录 B.2]

一个很容易遗漏的细节是：**通信 kernel 也会占 SM、HBM 带宽等资源**。它不在主计算流上，不等于它免费。单独 benchmark 中最快的通信实现，未必是与 GEMM 并发时总时间最短的实现。

5.2 为什么不能直接让同一 microbatch 的计算盖住通信?

对于同一个 microbatch，存在硬依赖：

$$D(m) \to E(m) \to C(m).$$

专家必须等 dispatch 的输入，combine 必须等专家输出。如果只把这三段放进不同 stream，依赖仍然存在。想增加重叠，需要找到**其他 microbatch 或其他反向子任务中的独立工作**。[S1，§4.2.3]

报告利用 1F1B 调度，把一个 microbatch 的前向与另一个的反向交错。与同时保留两个新前向相比，这能更好利用原本已存在的激活生命周期，但并不是说所有重叠配置都不增加显存。

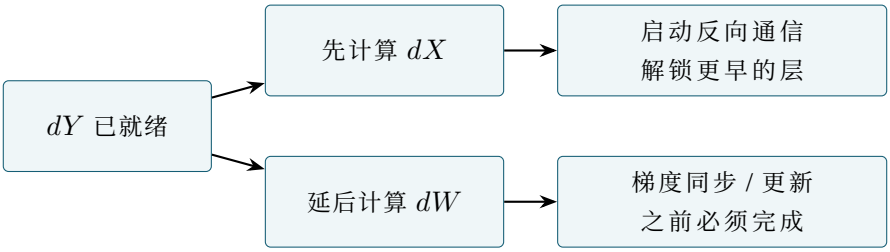
5.3 dgrad/wgrad 分离：理解这篇最关键的调度技巧

对于线性层 $Y = XW$：

$$dX = dY\, W^{\mathsf{T}}, \qquad dW = X^{\mathsf{T}} dY.$$

dX 要尽早交给前一层或反向通信，影响反向关键路径；dW 是本层的权重梯度，只要在对应梯度同步和 optimizer step 之前完成，就有更多调度余地。

因此可以把原来的整体 backward 拆成：**先算关键的 dgrad，启动相关通信，再用延后的 wgrad 和其他 microbatch 的前向填通信窗口**。这不是跳过权重梯度，而是推迟不必立即完成的部分。[S1，§4.2.3；S3]



优先解锁上游依赖；延后的 dW 可用于填充独立通信窗口。

忽略资源争用，单个窗口可以理想化为：

$$t_{\text{window}} \approx \max(t_{\text{comm}}, t_{\text{independent compute}}) + t_{\text{boundary}}.$$

现实中还会受到 SM carve-out、HBM 争用、PP warmup/flush 和通信偏斜影响。报告展示的高 overlap 比例，描述的是特定配置下被隐藏的通信份额，不能转写成 “所有训练任务快了同样比例”。

这部分的真正贡献是依赖图调度，而不是 stream 数量。 Kernel 优化也应测并发场景，而不只是测空闲 GPU 上单个算子耗时。

六、计算效率：Grouped GEMM 和 CUDA Graph 各解决一层问题

6.1 Grouped GEMM：让许多不一样的矩阵共享调度机会

每个专家处理不同的 M_e，专家权重也不同。Grouped GEMM 将这些独立问题联合调度，减少反复启动和单组矩阵的尾部浪费。它不是把所有专家视作一个共享权重的大矩阵，也不是单靠 batched API 名称就能得到同样性能。[S1，§4.3.2]

当专家更细、M_e 更小，tile 利用率与不同专家的负载偏斜就更重要。Fusion 则处理另一类开销：router、索引/offset、permute、unpermute、激活、量化等很多操作主要花时间在读写和启动上。[S1，§4.3.2–5]

从优化角度，应该同时看有用 FLOPs、padding FLOPs、访存、host launch、同步和尾部。在只报告总 TFLOPS 的 benchmark 中，不能允许大量 padding 计算把 “做了更多无用工作” 包装成效率提升。

6.2 为什么 FP8 越快，CPU 反而越容易成为瓶颈?

假设 GEMM 耗时缩短，而 Python 调度、launch 和 shape 同步没有变，它们在 step time 中的占比就会上升。这是一个简单的 Amdahl 型现象：**计算加速会暴露原来被掩盖的控制开销**。

CUDA Graph 通过捕获并重放执行图，减少逐个 kernel 的 host launch 开销。它并不自动减少模型 FLOPs，也不能消除真实的数据依赖。[S1，§4.3.6]

报告采用的实用过渡方案是 partial capture：先捕获 attention、router、preprocess 等可稳定执行的部分，动态 expert 路径暂时保持其他执行方式。这样的方案就可能有价值，不必等到全模型一次性 capture 成功才开始收益。[S1，附录 B.2]

七、动态 MoE 怎样进入 CUDA Graph：三个问题，三层答案

7.1 动态 shape 的真正矛盾在哪里？

Dropless MoE 的 M_e 由 GPU 上的 router 产生。如果 host 为了分配精确大小的 buffer、构建 Grouped GEMM 参数或决定 launch 配置而读取这些计数，就会引入 device-to-host 同步。CUDA Graph 又需要稳定的地址与可重放的执行结构。[S4；S5]

因此，问题不是 “GPU 不能处理动态数据”，而是：**动态计数是否必须回到 host 才能继续执行？为避免回 host 预分配的容量，又是否浪费了太多显存？**

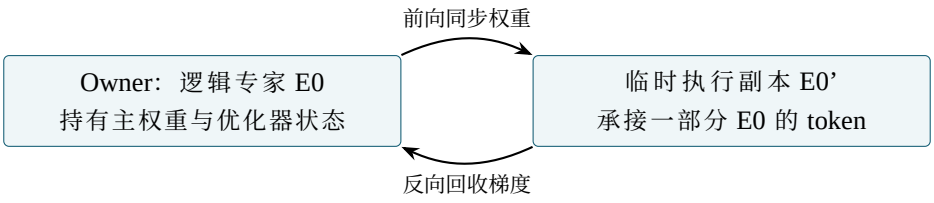
第一层答案是让 shape/计数保持在 device，kernel 从 device metadata 读取真实工作量；host 仍然可以重放静态 launch 结构。论文所说 device-initiated Grouped GEMM，不应草率理解成一定使用 CUDA Dynamic Parallelism 在 GPU 内部再启动 kernel。[S4]

7.2 ECHO：复制热门专家的执行位置，不是改变逻辑路由

ECHO（Elastic Cloning for Hot Experts）试图缓解不同 EP rank 接收量差异：把热门专家临时复制到低负载 rank 的空闲槽位，让一部分 token 去副本计算。[S1，§4.3.7；S4]

可以用一个简化例子理解：GPU A 上专家 E0 收到 80 份工作，GPU B 只收到 20 份。给 B 临时放一个 E0 副本，将部分 E0 工作转过去，可能缩短等待 A 的尾部。

但训练中的副本必须满足：前向权重与原专家一致；反向副本梯度回收到原专家；optimizer 与 checkpoint 所有权保持清晰。**它不是新增一个独立学习的专家，也不是偷偷改变 token 的 top-k 专家选择**。[S4]



执行位置改变；逻辑专家身份与训练参数所有权不改变。

它的净收益可用一个**解释性不等式**思考：

$$\Delta t_{\text{tail}} > t_{\text{weights}} + t_{\text{grad return}} + t_{\text{planning}} + t_{\text{extra}}.$$

只有消除的长尾超过复制、归约与管理成本，才真正有利。它与辅助负载均衡 loss 的区别在于：后者影响模型如何选择专家，前者主要调整既定逻辑选择的物理执行位置。

7.3 Paged Stashing：计算缓冲与反向保存区分离

为了 capture，可以为专家计算预留最坏情况的大 buffer。但若每一层、每一个尚未反向的 microbatch 都长期占着最坏情况 buffer，显存会迅速膨胀。[S5]

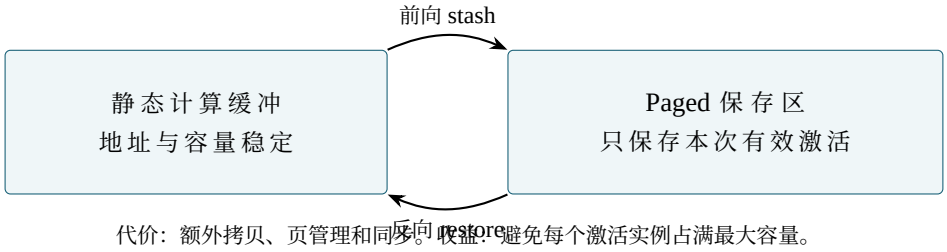
Paged Stashing 引入一个间接层：前向用稳定地址的大计算缓冲；计算完成后，把有效激活搬进按实际需求使用的 page；反向再把它们 restore 回计算缓冲。Page 分配与回收在 device 上管理，以适应 PP/VPP 下复杂的生命周期。[S5]

简化表达两种占用：

$$M_{\text{naive}} \approx N_{\text{live}} M_{\max},$$

$$M_{\text{stash}} \approx M_{\text{workspace,max}} + \sum_{j \in \text{live}} M_{\text{actual},j} + M_{\text{metadata}} + M_{\text{overlap}}.$$

这里 N_{live} 指尚未释放的激活实例数，可能同时受层数与在途 microbatch 影响。第二个式子的 overlap 项明确计入额外并发缓冲，避免 “共享一个 buffer 就没有任何额外开销” 的误解。



代价：额外拷贝、页管理和同步。收益：避免每个激活实例占满最大容量。

它与 KV Cache 分页有相似的间接寻址思想，但对象不同：**这里主要解决训练激活的跨前后向保存，不是把 decode 的 KV Cache 直接搬进训练**。

7.4 方法、实现、主实验，必须分开

截至本次核对，论文指向的 ECHO PR #2368 仍为 open/draft，说明中称其为研究原型；Paged Stashing PR #2690 则显示已经合并到 dev。这里只是在描述这两个 PR，**不能由 ECHO 旧 PR 未合并推断仓库内不存在其他相关实现**。[S4；S5]

最重要的实验边界独立于 PR 状态：主表对应附录列出的 GB200/GB300 图执行 scope 是 attention、router、preprocess；H100 对应配置未开启图执行。因此，**不能把主表的 1,233 TFLOPS/GPU 归功于 ECHO + Paged Stashing 的完整动态 MoE 图方案**。[S1，附录 B.2]

八、FP8 与 NVFP4：精度、布局、通信必须一起看

8.1 “更小的 dtype” 只是开始

报告讨论三类不同路径：Hopper 的 blockwise FP8，Blackwell 的 MXFP8，以及 NVFP4。它们的缩放粒度、scale 格式、GEMM 布局要求和量化方式不同，不能把它们都简化成一次 cast。[S1，§5]

路径	报告讨论的主要特点	需要特别关注的成本
Blockwise FP8	例如激活按 1×128、权重按 128×128 分块	分块 amax、scale、重排、前后向不同操作数
MXFP8	1×32 微缩放块、E8M0 scale	row/column 两种量化视图、scale 布局
NVFP4	E2M1 数值、分层 scale；结合 RHT、随机舍入等	分组量化、Hadamard、amax、padding、布局融合

表中是该报告讨论的训练 recipe，不是对所有库和未来版本的统一约束。Router、归一化、主权重、梯度累加等对象也不应默认全降到最低精度。

8.2 为什么量化后的矩阵不能总是直接转置?

若量化 scale 按行上的连续 block 计算，转置会改变哪些元素属于同一个 block。因此一般有:

$$Q_{\text{row}}(X^{\mathsf{T}}) \neq \left(Q_{\text{row}}(X)\right)^{\mathsf{T}}$$

这里 Q_{row} 始终表示沿同一行方向分块的量化；转置后重新分块的结果，一般不同于转置原先的量化结果。两边相等需要额外条件。

更直接地说：前向与反向需要的两个操作数布局，可能必须各自量化。额外视图会消耗存储、转换时间甚至通信。**GEMM 理论峰值提高，不代表整个低精度数据路径按同样比例加速**。[S1，§5.1–5.2]

8.3 FP8 不保证所有通信减半

这是报告中特别值得注意的细节。某些 FP8 primary-weight 路径可以减少参数通信；但 §5.2.2 讨论的 MXFP8 参数 gather 仍采用临时 BF16，因为分别通信 row/column 两份 FP8 并不自然比一份 BF16 更省。NVFP4 两种视图的相应路径，报告讨论的是相对 BF16 约 50% 的通信节省，而非看到 4-bit 就推出 75%。[S1，§5.2.2]

因此需要逐段标注：本地保存什么 dtype? 网络上传什么 dtype? 传几份? 接收后是否重新量化? 专家 dispatch 与 DP 参数 AllGather 是完全不同的通信，不要混在一个 “FP8 减通信” 结论里。

8.4 小 M、padding 和量化开销会改变答案

某些 grouped kernel 与 scale 布局需要将每专家 token 数对齐到较大粒度；报告讨论的部分实现需要 128 对齐。这是具体 kernel 数据布局要求，不是所有 FP8/FP4 GEMM 永久固定的数学要求。[S1，§5.3]

设对齐粒度为 a:

$$M_{\text{pad}} = a\left\lceil M/a \right\rceil, \qquad \eta_{\text{useful}} = \frac{M}{M_{\text{pad}}}.$$

人为例子: $M = 17, a = 128$ 时，容量膨胀约 7.53 倍，有效行比例仅 13.3%。实际 kernel 能否跳过空工作，决定计算是否也付出同等比例；不能仅凭容量比例断言延迟一定扩大 7.53 倍。

对小 M 场景，应把 quantize、scale/amax、padding、permute、GEMM 和还原全部纳入时间。只有大训练 batch 下的 Grouped GEMM 数字，不能证明 batch=1 decode 的专家也有同样收益。

8.5 精度收益要分三层验证

第一层是算子前后向误差；第二层是路由与训练 step 的误差传播；第三层是长程收敛和最终质量。尤其 top-k 边界附近，较小的 score 差异可能改变离散路由结果。

报告主吞吐表包含 BF16 与 FP8，但不包含同等口径的 NVFP4 全模型结果。**“支持 NVFP4” 不能直接推成 “主表已经证明 685B 模型 NVFP4 全程收敛与最佳吞吐”**。[S1，表 11、§5]

九、长上下文与 RL：瓶颈会换位置

9.1 长序列时，不要继续假设 Expert 永远最重要

固定模型下，MLP/专家的算术工作量大致随 token 数线性增加；标准 dense attention 的主要矩阵计算随序列长度近似二次增长。FlashAttention 类技术减少中间矩阵存储与访存，但不等于把完整 dense attention 的算术复杂度变成线性。[S1，§6]

这会改变重计算策略：短序列里看起来便宜的模块，在长序列下可能成为昂贵大头。报告强调长上下文时重新评估 core attention 重计算，结合 CP/TP 分片、offload 和更合适的选择性重算，而不是沿用短上下文的全部开关。

9.2 Token 数相等，不等于 attention 工作量相等

Packed sequences 保留独立样本边界，因此可以避免把所有样本 pad 到最长样本。但 attention 的算术工作量更接近：

$$C_{\text{attention}} \propto \sum_i s_i^2,$$

而不是只由 $\sum_i s_i$ 决定。[S1，§6.4]

例如，总 token 都为 16K：一条 16K 序列对应 $16^2 = 256$ 个归一化工作单位；四条 4K 序列是 $4 \times 4^2 = 64$，只相差组织方式，attention 工作量就可差约四倍。这里忽略线性项和具体 causal 常数，是为了说明 packing 的调度原则。

因此，仅按总 token 均分给 DP ranks，仍可能出现严重同步等待。**正确的负载估计应同时考虑线性 token 成本、attention 的二次项和显存约束**。

9.3 Dynamic CP：动态改变切分，不动态搬整套参数

Dynamic CP 按 microbatch 选择合适的 CP size，并与 packing 计划联合调度。核心便利在于：CP 改变 token 切片与 attention 通信组，不要求为每个 microbatch 重新迁移模型参数和优化器状态。[S1，§6.4.2]

实现会预建候选 CP groups，运行时选择；同时传播动态 microbatch 数、序列边界等元数据，确保 PP 各阶段一致。规划既要控制峰值显存，又要均衡工作量，而不是永远选最小 CP。

对于变长 batch，loss/梯度归一化还必须与任务目标一致。报告的 packed-token 训练按有效 token 归一化；把按 token 加权目标误写成 “每个 microbatch 的平均 loss 再平均”，会在长度不同时改变样本权重。[S1，式 3]

9.4 Router Replay 保证什么，不保证什么？

报告还讨论与 RL 训练生态的连接，包括训练与 rollout 的权重转换、资源管理和 Router Replay。训练、推理 backend 的数值路径差异，可能使同一输入落到不同专家；记录并重放路由可以对齐逻辑专家路径。[S1，§10]

但必须区分：**逻辑路由一致、浮点逐位一致、完整策略概率一致**是三个不同目标。重放专家索引不能自动保证 GEMM、归约、量化、softmax 或最终 logprob 全部一致，也不能凭此证明所有 off-policy 问题被解决。

在 RL 系统里，真正应计时的是：

$$t_{\text{cycle}} = t_{\text{rollout}} + t_{\text{transfer}} + t_{\text{train}} + t_{\text{weight sync}} + t_{\text{other}}.$$

只把训练后端提速，而 rollout、样本处理或权重同步没有变，整个闭环的加速可能很小。这是从本文训练系统走向 RL Infra 时需要补上的评价维度。

十、主实验：数字很强，但有三个不可省略的条件

10.1 Table 11 的原始口径

以下为作者主表结果；**所有配置均使用 force-balanced routing，单位为每 GPU 吞吐，未在本解析中独立复现**。[S1，表 11]

模型 / 系统	GPU 数	序列长度	精度	TFLOPS/GPU	tokens/s/GPU
DeepSeek-V3 / GB300	256	4,096	MXFP8	1,233	4,730
DeepSeek-V3 / GB200	256	4,096	MXFP8	1,048	4,020
DeepSeek-V3 / GB200	256	4,096	BF16	857	3,298
DeepSeek-V3 / H100	1,024	4,096	FP8-BLK	368	1,412
Qwen3-235B / GB300	256	4,096	MXFP8	974	6,583
Qwen3-235B / GB200	256	4,096	MXFP8	919	6,212
Qwen3-235B / GB200	256	4,096	BF16	750	5,100
Qwen3-235B / H100	256	4,096	BF16	320	2,132
Qwen3-235B / GB300	128	131,072	MXFP8	1,150	1,556

这些数字证明的是，在列出的模型形状、硬件、并行配置和均衡路由条件下，系统能持续提供相当高的吞吐。它不是单个 kernel 的性能，也不是完整训练到某种质量目标的成本对比。

10.2 条件一：均衡路由不代表真实负载

Forced balance 有合理用途：它降低工作负载偏斜，让某些框架与硬件对比更可控。但它也绕开了真实 router 的一个重要困难：热门专家与长尾 rank。

因此，主表无法单独回答：训练早期路由不均时多慢？某些输入分布集中到少数专家时显存峰值怎样变化？ECHO 在这种负载下净赚多少？需要额外的自然路由和可控偏斜实验。

我不会因此否定主表，而会把它归类为：**均衡工作负载下的系统吞吐基准，而非对所有实际训练阶段的速度保证**。

10.3 条件二：FP8 对比不只是 dtype 改了

DeepSeek-V3 在 GB200 上，MXFP8 相对 BF16 的 tokens/s/GPU 增幅约为：

$$\frac{4020}{3298} - 1 \approx 21.9\%.$$

但附录 B.1 的两组配置是：[S1，表 20]

配置项	GB200 MXFP8	GB200 BF16
TP	1	1
PP	4	8
EP	64	32
VPP	4	4
Microbatch size	1	1
Global batch size	8,192	4,096

这可以支持 “各自调优后的 FP8 栈更快”，但不能支持 “仅将 BF16 GEMM 改成 FP8，固定其余所有条件，得到 21.9% 提升”。更好的精度可能通过省内存使并行配置改变，这是有效的系统收益；**只是在进行因果归因时，必须明确它不是单因素消融**。

10.4 条件三：单卡吞吐不等于整集群吞吐

DeepSeek-V3 的 GB200/H100 每卡 token 吞吐比为 $4020/1412 \approx 2.85$。但对应卡数分别是 256 和 1,024。按主表口径相乘，整集群 token 吞吐分别约为 103 万和 145 万 token/s。

这个计算不是一项新实验，只是避免误读 “快约三倍” 的分母。不同卡数、batch、硬件与成本下，应分别讨论每 GPU 效率、总吞吐、单 step 时延、能耗和费用。单个 1,024 卡成功点也不等于完整证明强扩展或弱扩展接近线性。

长上下文同样要谨慎：Qwen 128K 的 TFLOPS/GPU 高于部分 4K 结果，不代表处理一个 token 更便宜。二次增长的 attention 工作恰好可能更容易发挥 Tensor Core 效率，使 MFU 上升而 token 吞吐下降。

十一、严谨性与可复现性：我会追问什么？

11.1 一个具体的配置不一致：H100 的 PP 到底是 4 还是 8?

当前 arXiv v2 HTML 中, 表 17 列出 H100 DeepSeek-V3 为 **TP2 / PP8 / EP64**; 附录表 20 相同 368 TFLOPS/GPU 结果列出的 PP 却是 **4**。[S1，表 17、表 20]

本次未能用原 PDF 的表格图像排除 HTML 转换问题，因此将它列为**需要用原 PDF 与对应 launcher 进一步核对的文档差异**，不擅自断言哪一个正确，更不能把它解释成实验造假。

差异具有实质影响。若总卡数为 1,024，CP1、ETP1，则：

$$PP8:\ DP = 64,\ EDP = 2; \qquad PP4:\ DP = 128,\ EDP = 4.$$

这会改变每卡层数、pipeline 行为、专家副本组和优化器分片。复现时不应照抄其中一个表格后，把对不上的结果全部归因于硬件波动。

11.2 FP8 节省的是哪些激活?

报告 §4.1.3 明确给出：上述 131 GB activation 里，大约 32 GB 的线性层输入适合 FP8 保存，减半后省约 16 GB，即整体约 12%。这比泛泛说 “FP8 把 activation 内存减半” 准确得多。[S1，§4.1.3]

需要区别三个分母：一个 eligible tensor 的字节数、全部 activation、整个训练显存。即便 dtype 从 BF16 变 FP8，也不能把整台卡的占用直接除以二。

11.3 四个本地专家仍然需要组织 token

正文案例中，EP64 对 256 专家意味着每 GPU 4 个专家；附近文字又称因此消除本地 token permutation。一般情况下，多个本地专家仍需要按专家组织 token。某个 backend 可以把它融合进 dispatch 或直接写成目标布局，但不能从 “4 个专家” 这一计数本身推出没有布局整理。[S1，§9.2.2；S3]

更准确的表述应是：**可能消除了一个独立 permutation 阶段，而不是消除了逻辑上的按专家分组**。

11.4 有组件实验，不等于所有收益都完成了独立归因

报告包含显存分解、局部优化和通信等测量，因此不能简单说 “完全没有消融”。但主表是一整套最优配置的合成结果，不能把它的提升百分比分摊给每个技术，也不能认为所有功能都在每个平台同时打开。

尤其附录明确区分了 GB200 的图执行与 H100 的通信重叠设置。**优化功能是候选工具，不是越多开关一起开越快**。[S1，附录 B.2]

11.5 缺少哪些证据，会限制什么结论?

想进一步主张的结论	还需要的证据
真实 router 也有主表一样的吞吐	自然路由、不同 skew、峰值显存与慢 rank 统计
FP8 单独贡献某个提升比例	固定并行度、batch、dispatcher 的单因素对照
ECHO 带来端到端收益	包含权重复制、梯度回收、规划成本的全步测试
NVFP4 全程训练质量可靠	等数据/等训练预算的长期收敛与最终评价
千卡扩展效率接近线性	多个卡数点、固定强/弱扩展定义、网络条件
训练系统整体更经济	能耗、费用、有效训练 token 与 time-to-quality

这是对证据范围的界定，不否定报告作为训练系统工程手册的价值。

十二、怎样把论文变成一个实际的优化项目?

12.1 第一原则：先固定要保持什么

对同一个优化，可以有三种不同的正确性标准：保持数学计算、在给定容差内一致、与基线逐位一致。三者的工程约束不同。尤其概率前移、FP8 量化、并发归约或副本梯度归并，可能保持训练数学语义，但改变浮点计算顺序。

建议在 benchmark 配置中写清楚：路由索引是否固定，概率是否固定；随机状态是否恢复；哪些张量用 bitwise 比较，哪些用误差界；是否检查参数更新后的差异。**只看 forward 输出 close，并不足以证明 backward 与训练更新正确**。

12.2 不必从完整 685B 复现开始

下面是按问题分层的验证路线，不是论文已经给出的实验承诺：

层级	最小对象	主要指标	能回答 / 不能回答
单卡算子	Grouped MLP、permute、量化与概率融合	误差、峰值内存、总算子链时延	能解释 kernel 成本；不能证明跨节点扩展
单机多卡	1–数层 MoE、EP2/4/8	exposed A2A、并发 GEMM、负载偏斜	能验证机内调度；不能代表 IB 网络
跨节点	固定模型片段与真实 dispatcher	最慢 rank、网络拥塞、SM 争用	能测试拓扑与 overlap；不等于训练质量
小模型训练	保持相同算子/路由机制的缩小模型	loss、梯度、optimizer 更新	能排查数值问题；不直接证明 685B 收敛
端到端	固定配置的真实训练 / RL 循环	step、tokens/s、P95/P99、质量	用于确认微基准收益没有被其他阶段抵消

12.3 三个与你的关注点直接衔接的课题

课题 A：确定性约束下的概率前移与 fused backward。固定输入、专家索引、概率、权重和梯度，比较标准路径与前移路径；分别测 BF16、FP32 累加与 FP8 recipe。重点不是重新发明恒等式，而是研究保存张量、融合、舍入位置和权重更新误差之间的边界。

课题 B：真实 skew 下的分组 GEMM 与通信资源配额。使用均匀、可控偏斜以及脱敏真实 histogram，扫描通信 SM 预算、tile、grouped 调度和 overlap。报告最慢 rank 的整步时间，而非只报告平均专家 GEMM；固定总有效 token，避免 padding 与重复路由把吞吐数字抬高。

课题 C：动态 shape 的 device metadata 与保存区管理。比较 eager、partial graph、device-count grouped kernel，以及有/无紧凑保存区的内存曲线。分别测 host sync、峰值 reserved/allocated memory、stash/restore 拷贝和 replay latency。ECHO、Paged Stashing 与新的虚拟专家功能存在版本和兼容性差异，不能随意拼接 flags。[S3–S5]

这些是基于报告与公开实现提出的实验候选，不是对创新性、录用概率或固定完成时间的保证。

12.4 一个比 “开关全开” 更可靠的调优顺序

先记录硬件拓扑、软件 commit 与精度 recipe，建立能运行且数值检查通过的基线；再定位显存峰值来自权重、优化器还是 activation；用较低开销的内存优化扩展可行布局；选择使主要 EP/TP 通信尽量留在高速域的布局；最后分别处理暴露通信、host 空隙和 kernel 效率。每轮修改后重新 profile，因为瓶颈可能已经换位置。[S1, §9；S3]

运行时至少记录：有效 tokens/s、step latency 分布、每专家 M_e 分布、padding 比例、最慢 rank、exposed A2A、GEMM 与通信并发时长、峰值 allocated/reserved memory、量化/转换时间和训练更新误差。

最危险的捷径是把微基准里最快的 kernel、最快的 dispatcher、最省显存的策略直接拼起来，并假定结果仍然最好。这些组件共享资源和依赖，组合后的关键路径才是优化对象。

十三、最后把整篇论文收束成一个系统观点

这篇报告最重要的不是某个 headline TFLOPS，而是五个设计原则。

让布局适应子模块。 Parallel Folding 将 Attention 的 TP/CP 需求与专家的 ETP/EP 需求解耦，避免一种布局同时伤害两边。

先问张量为什么必须活着，再问它如何压缩。概率前移、output-discarding recomputation 和 Paged Stashing 都是在重写生命周期，而不只是减小 dtype。

重叠的是独立工作，不是名字叫 stream 的队列。 dgrad/wgrad 分离与跨 microbatch 1F1B，价值在于重排依赖，找到可填充通信窗口的计算。

动态数据不必意味着 host 每步做动态决策。 Device-side metadata、稳定计算缓冲与紧凑保存区，让动态逻辑尺寸和静态执行结构有机会共存。

高吞吐要附带负载、精度和配置条件。强制均衡、FP8 recipe、NVLink 域、PP/EP 与 batch 都影响比较的含义，不能只截一个 TFLOPS 数字。

用一句话概括：

> **MoE 的稀疏性只提供了潜在计算优势；真正把优势兑现为训练速度，需要共同设计参数放置、激活生命周期、通信依赖、动态内存与 GPU 执行。**

与你熟悉的 Tile/kernel 优化相比，这篇论文把观察尺度向上推了一层：不只问 “这个 GEMM 为什么慢”，还要问 “为什么它只能拿到这种形状、为什么输入还没到、为什么一个中间结果要保存这么久，以及为什么 GPU 算快后反而在等 CPU”。

附录 A：术语速查

术语	本解析中的含义
TP / ETP	稠密模块 / 专家内部的张量并行
DP / EDP	稠密模块 / 同一专家副本的数据并行
CP	将上下文 token 分片，配合 attention 通信
PP / VPP	层级流水线 / 虚拟流水线分块与交错调度
Dispatch / Combine	专家输入发送与组织 / 专家输出返回与聚合
dgrad / wgrad	对层输入的梯度 / 对权重的梯度
Dropless	不因为容量上限而丢弃已路由 token
ECHO	将热门逻辑专家临时复制到其他执行位置
Paged Stashing	将跨前后向保存的有效激活放进页化保存区
MXFP8 / NVFP4	具体的低精度数值与缩放 recipe，不只是 dtype 名称
Exposed communication	未被独立计算覆盖、实际延长关键路径的通信
MFU	模型有效 FLOPs 相对设备峰值能力的利用指标，需统一口径

附录 B：来源与核对范围

以下均为原始论文、官方仓库或作者实现资料。报告中的图是为解释方法重新绘制的结构示意图，不是原论文测量曲线；数值例子已标出假设。未独立运行分布式训练或重新测量主表吞吐。

[S1] 主报告。 Zijie Yan 等，*Scalable Training of Mixture-of-Experts Models with Megatron Core*，arXiv:2603.07685v2。重点核对 §2–6、§8–10、表 3/4/11/17/20 与附录 B.2。正文与附录阅读基于 arXiv HTML；表 17/20 差异需要再用原 PDF 与 launcher 排除格式转换因素。

https://arxiv.org/abs/2603.07685

https://arxiv.org/html/2603.07685v2

[S2] Parallel Folding 独立论文。Dennis Liu 等，*MoE Parallel Folding: Heterogeneous Parallelism Mappings for Efficient Large-Scale MoE Model Training with Megatron Core*，arXiv:2504.14960。用于核对两套并行网格与 token dispatcher 的前后向逻辑，不把其旧模型 MFU 与主报告新平台数字混用。

https://arxiv.org/abs/2504.14960

[S3] NVIDIA/Megatron-LM 官方 MoE README。本次读取模型支持、并行布局、dispatcher、内存/通信优化等部分。main 文档可能持续变化，因此不能用当前 README 替代论文实验的固定 commit。

https://github.com/NVIDIA/Megatron-LM/blob/main/megatron/core/transformer/moe/README.md

[S4] ECHO PR #2368。用于核对临时专家副本、device-side shape、权重/梯度一致性与原型状态。本次查询显示 open、draft，PR head 为 `a2b16b8733bb4ced46880d6adee6f19124732991`。这不排除其他路径存在后续实现。

https://github.com/NVIDIA/Megatron-LM/pull/2368

[S5] Paged Stashing PR #2690。用于核对计算缓冲/保存区分离、页管理及 stash/restore 调度。本次查询显示 merged，目标分支 dev；合并提交 `c0c4fdc45e9f0f8047e29f2cb8669613169752c3`。

https://github.com/NVIDIA/Megatron-LM/pull/2690

复现入口（原文给出，需自行固定版本与配置）：

https://github.com/NVIDIA-NeMo/Megatron-Bridge/tree/main/scripts/performance

https://github.com/yanring/Megatron-MoE-ModelZoo

研究阅读笔记·不等同于作者认证的复现结果。

解读重点：方法、约束、实验分母，以及对实际 AI Infra 优化的可验证启示。